[1] Author(s), *Book Title*, Edition. City: Publisher, Year.Project Interim Report

On

File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By –

Rahul

Reg No. - RU-25-11076

Bachelor of Technology in Computer Science & Engineering in Association with Google

Under the Guidance of

Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

The **Smart Home Automation System** is a Core Java-based project designed using Object-Oriented Programming (OOP) principles to simulate and manage smart devices within a home environment. The project models real-world home appliances such as lights, fans, security systems, and temperature controllers as objects, allowing users to control them through a centralized interface.

This system is needed to improve convenience, energy efficiency, and security in modern homes. With increasing dependence on technology, automating daily household tasks reduces manual effort and helps users manage devices remotely or automatically based on predefined conditions.

The system performs key functions such as turning devices on/off, scheduling operations, monitoring device status, and triggering alerts for security events. It uses OOP concepts like encapsulation, inheritance, and polymorphism to create a modular and scalable design. For example, a base “Device” class can be extended into specific device types, making the system easy to expand.

Overall, the project demonstrates how Core Java and OOP concepts can be applied to build a practical, efficient, and user-friendly home automation solution.

Introduction

The **Smart Home Automation System** is a Core Java-based application developed using Object-Oriented Programming (OOP) concepts to simulate a modern automated home environment. In this project, different home appliances such as lights, fans, air conditioners, and security systems are represented as objects, allowing users to control and monitor them through a centralized system.

This project is needed to enhance convenience, improve energy efficiency, and reduce human effort in managing daily household activities. With the growing demand for smart living, automation systems help users control devices easily, save electricity by managing usage, and ensure better home security.

The system performs several important functions such as turning devices on and off, scheduling operations, monitoring the status of appliances, and generating alerts for unusual activities like security breaches. It also allows users to manage multiple devices efficiently from a single interface.

Objectives / Features of the Project:

- Manage and control smart home devices
- Track the status of connected appliances
- Reduce manual work through automation
- Improve energy efficiency and security
- Provide scheduling and alert functionalities
- Apply OOP concepts like encapsulation, inheritance, polymorphism, and abstraction
-

Scope

This project is suitable for smart homes of small to medium scale and can be further enhanced with advanced technologies like database integration and IoT connectivity in the future. By integrating a database, the system can store device history, user preferences, and activity logs more efficiently. Additionally, future extensions may include mobile app control, voice assistant integration, and real-time monitoring through the internet, making the system more scalable and practical for real-world applications.

System Requirements

Hardware:

- Computer/Laptop with minimum 4GB RAM
- Basic processor (Intel i3 or above recommended)
- Keyboard and mouse for user interaction

Software:

- Java Development Kit (JDK)
- IDE such as VS Code, Eclipse, or Edit Plus
- Operating System: Windows/Linux/macOS

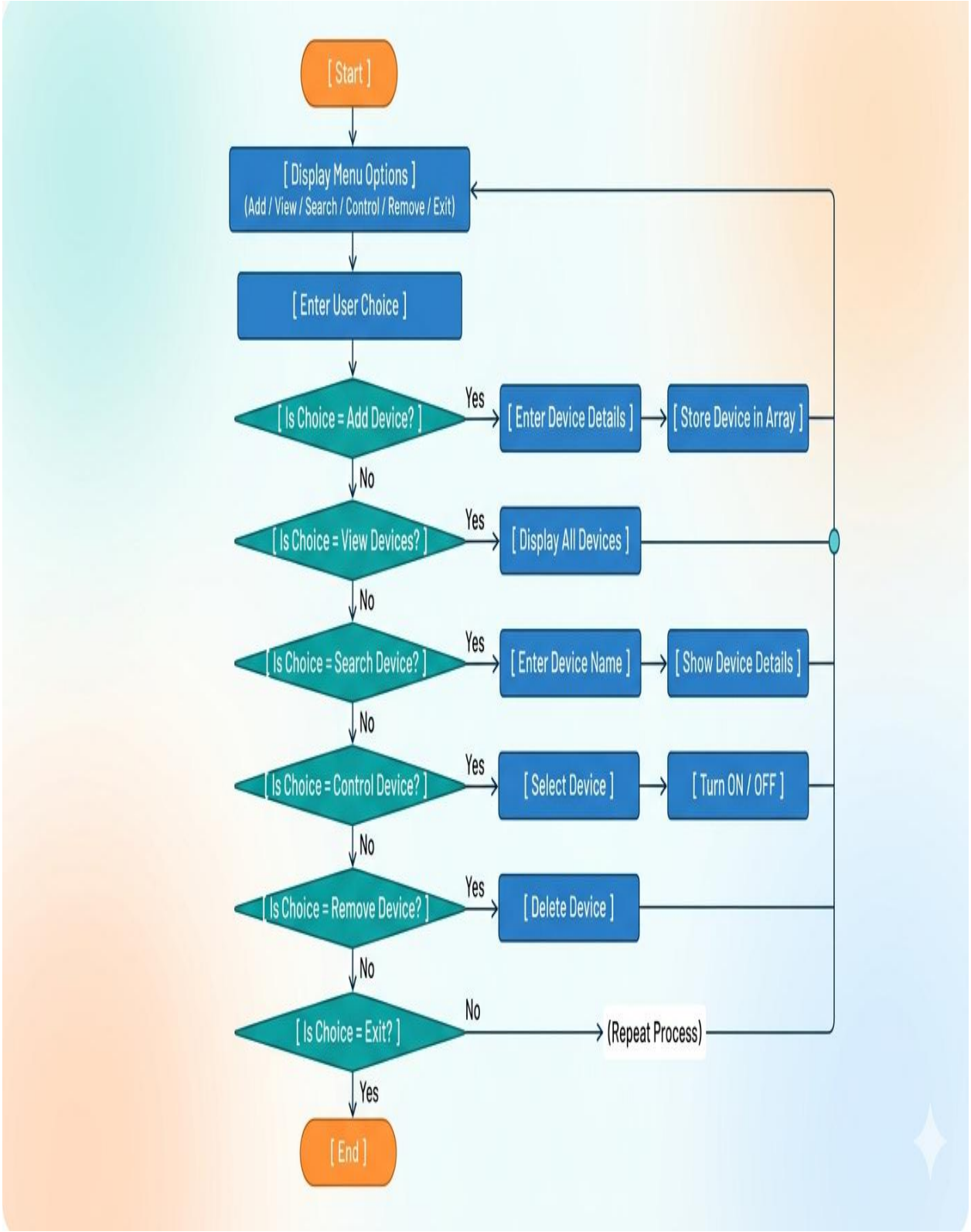
Methodology

Working of the Smart Home Automation System

The system works in a step-by-step manner where the user interacts with the application to manage and control smart devices efficiently:

1. **Add Devices:**
   Initially, the user adds different smart devices (like lights, fans, AC, security system) into the system. Each device is created as an object with properties such as device name, status (ON/OFF), and type.
2. **View Devices:**
   The system displays the list of all added devices along with their current status. This helps the user keep track of all connected appliances.
3. **Search Devices:**
   Users can search for a specific device using its name or type. This makes it easy to quickly find and control a particular appliance.
4. **Control / Modify Devices:**
   The user can modify device states, such as turning a device ON or OFF, or updating its settings. This simulates real-time control of smart home devices.
5. **Schedule Operations (Optional Feature):**
   The system may allow users to set schedules for devices, such as turning lights on/off at specific times.
6. **Monitor Status:**
   The system continuously keeps track of each device's status and reflects any changes made by the user.
7. **Remove Devices:**
   If a device is no longer needed, the user can remove it from the system.

Flowchart


[Start]
[Display Menu Options]
(Add / View / Search / Control / Remove / Exit)
[Enter User Choice]
[Is Choice = Add Device?]
Yes
[Enter Device Details]
[Store Device in Array]
No
[Is Choice = View Devices?]
Yes
[Display All Devices]
No
[Is Choice = Search Device?]
Yes
[Enter Device Name]
[Show Device Details]
No
[Is Choice = Control Device?]
Yes
[Select Device]
[Turn ON / OFF]
No
[Is Choice = Remove Device?]
Yes
[Delete Device]
No
[Is Choice = Exit?]
No
(Repeat Process)
Yes
[End]

Implementation

1. Explanation of Different Classes (Logic Only)

Device Class

This class represents each smart device in the system. It contains properties such as device name, type, and status (ON/OFF). The main logic of this class is to manage individual devices. It allows operations like turning a device ON or OFF and displaying its current state. Each device works independently using this class.

Smart Home System Class

This class acts as the core controller of the system. It manages multiple devices using an array or list. The logic includes adding new devices, displaying all devices, searching for a specific device, updating device status, and removing devices. It connects user actions with actual device operations and ensures smooth system functionality.

Main Class

This is the entry point of the program. It handles user interaction by displaying menu options and taking input. Based on the user's choice, it calls the appropriate methods from the SmartHomeSystem class. It also runs a loop so the system continues until the user exits.

---

2. Screenshot of Terminal

- Run your program in your IDE (VS Code / Eclipse)
- Perform operations like Add, View, Control
- Take a screenshot of the output screen
- Paste it into your project report

---

3. Future Enhancements / Improvements

- Add **database support** (MySQL) to store device data permanently
- Create a **GUI interface** using Java Swing/JavaFX
- Enable **mobile app control**
- Add **voice command support**
- Implement **IoT-based real device integration**
- Add **user login and authentication system**
- Introduce **automation rules** (e.g., auto ON/OFF based on time)
- Improve UI/UX for better user experience

Gantt Chart

1. Planning & Structure

A Gantt chart helps you break the project into tasks (design, coding, testing, etc.) and set timelines.
Even if it's already built once, rebuilding or modifying still needs planning.

2. Time Management

It shows:

- When each task starts
- When it should finish
- This helps avoid delays and last-minute panic.

Conclusion

The project successfully demonstrates the implementation of Core Java and Object-Oriented Programming (OOP) concepts such as encapsulation, inheritance, polymorphism, and abstraction. It provides a functional and user-friendly system for managing and controlling smart home devices. The project not only meets its objectives but also shows how real-world problems can be solved using structured programming techniques, making it a practical and effective solution.

References

Book

[1] Author(s), *Book Title*, Edition. City: Publisher, Year.

[1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

Website

Smartify - India's Leading Home Automation Brand